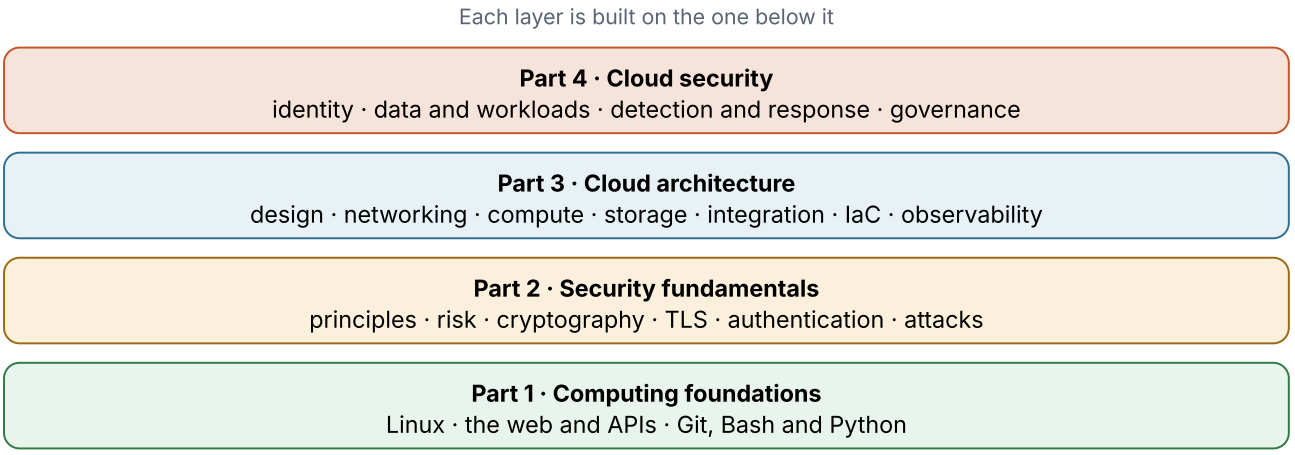
CLOUD SECURITY PATH · STUDY GUIDE 5

The Theory Book

The fundamentals behind every sprint, in one place: how computers, Linux and the web work; the principles of security; cryptography, certificates and login protocols; how cloud systems are designed; how the core AWS services really work; and the theory of cloud security. Each chapter has the big idea, the details, an analogy, a summary and a quick check.



Contents

0 How to use this book
1 Computers and operating systems
2 Linux for cloud engineers
3 How the web works: HTTP and APIs
4 Code skills: Git, Bash, Python
5 Core security principles
6 Risk and threat modelling
7 Cryptography
8 Certificates, PKI and TLS
9 Authentication and authorization
10 Common attacks
11 Designing cloud systems
12 Cloud networking in depth
13 Compute: VMs, containers, serverless
14 Storage and databases
15 Connecting services: queues and events
16 Infrastructure as code and CI/CD
17 Observability: logs, metrics, traces
18 Identity is the perimeter
19 Securing data and workloads
20 Detection and response in the cloud
21 Governance and compliance
22 Glossary and answers

0 How to use this book

> **This is the "why" book. Guide 4 tells you what to build each sprint; this book explains the theory underneath, so you can answer interview questions and solve problems you've never seen.**

Read the chapters that match your current sprint, a little each day. Don't read it cover to cover before you start building: theory sticks best when you use it the same week.

Sprint	Read these chapters
S1 · Launch	1, 2, 5
S2 · IAM + VPC	9, 12, 18 (first half)
S3 · EC2, S3, static site	3, 8, 13 (VMs), 14 (object storage)
S4 · Terraform	4, 16
S5 · Holiday	6, 7, 11 (an easy read for revision)
S6 · Serverless + Cognito	9 (tokens), 13 (serverless), 14 (databases), 15
S7 · Scaling, containers, logs	11 (scaling), 13 (containers), 17
S8 · Azure	Revisit 9, 12, 18 with the Azure columns of Guide 3
Phase B (Mar – Dec 2027)	10, 18–21, then the whole book again for interviews

How the guides fit together

Guide	Role
1 · Networking Fundamentals	Packets, ports, subnetting, firewalls, DNS. Chapter 12 here builds on it.
2 · Virtualization & Containers	Hypervisors, Docker, Kubernetes basics. Chapter 13 builds on it.
3 · Cloud Security Foundations	Every cloud service with AWS and Azure side by side; securing small organisations
4 · Sprint Workbook	What to build, step by step
5 · This book	**The theory behind all of it**

REMEMBER

For every concept, aim to answer three questions: What is it? Why does it exist? How can it go wrong? The third question is where security lives.

1 Computers and operating systems

> **Every cloud server is still a computer: a CPU running instructions, memory holding what's in use, storage keeping data, and an operating system sharing it all out safely.**

Part	What it does	In the cloud
CPU	Runs instructions. More cores = more work in parallel.	vCPUs on an EC2 instance
RAM	Fast, temporary memory for running programs. Lost on power-off.	Instance memory; Lambda memory setting
Storage	Slower, permanent data	EBS volumes, S3
Network card	Sends and receives packets	Elastic network interface (ENI)

What the operating system does

- **The kernel** is the core of the OS. It controls the hardware and decides which program runs when. Programs run in **user space** and must ask the kernel (through *system calls*) to open a file or send a packet.
- **Processes:** a running program. Each has an ID (PID), an owner (a user) and its own memory. You matched PIDs in Case 001 to spot duplicate alerts.
- **Permissions:** the kernel checks who owns a process before it touches a file. This is the operating system's own least privilege.
- **File systems** organise storage into files and folders (ext4 on Linux, NTFS on Windows).

THINK OF IT LIKE THIS

The kernel is a hotel manager. Guests (programs) can't walk into the kitchen or another guest's room. They ask the front desk (system calls), and the manager checks their key card (permissions) first.

Why this matters for security

- **Privilege escalation** means a process gaining more rights than it was given, for example from a normal user to root. Containing that is why services run as their own low-privilege users.
- Containers share the host's kernel (Guide 2), so a kernel weakness can affect every container on that host. VMs each have their own kernel.

QUICK CHECK (answers at the back)

1. What is the difference between kernel space and user space?
2. Why do services such as Wazuh run as their own user rather than as root?

2 Linux for cloud engineers

> **Most cloud servers, containers and security tools run Linux. You don't need to know every command, but you must be fluent in files, permissions, services, logs and networking.**

The file system layout

Path	Holds	You've used
`/etc`	Configuration files	`/etc/ssh/sshd_config.d/`, `/etc/chrony/`
`/var/log`	Logs	`/var/log/auth.log`
`/var/ossec`	Wazuh's files	`ossec.conf`, `alerts.json`
`/home`, `/root`	User home folders	`~/.ssh/`
`/usr/bin`	Installed programs	`nmap`, `ssh`

Permissions

`-rw-r----- 1 root wazuh ossec.conf` reads as: owner **root** can read and write; group **wazuh** can read; everyone else has nothing. In numbers: read = 4, write = 2, execute = 1, so `640` = rw- (6), r-- (4), --- (0).

Mode	Typical use
`600`	Private keys (`~/.ssh/id_ed25519`), credential files (`~/.aws/credentials`)
`644`	Normal readable files
`755`	Programs and folders

The essentials

Task	Commands
Services	`systemctl status|start|restart|enable`; drop-ins in `/etc/systemd/system/*.d/`
Logs	`journalctl -u NAME -b`, `tail -f FILE`, `grep`
Packages	`apt update && apt upgrade` (Ubuntu); `dnf` (Amazon Linux, Rocky)
Network	`ip a`, `ip route`, `ss -tulpn` (listening ports), `dig`, `curl`
Processes	`ps aux`, `top`, `kill PID`
Users	`id`, `sudo`, `useradd`, `/etc/passwd`

SSH keys in one paragraph

You keep a **private key**; the server holds your **public key** in `~/.ssh/authorized_keys`. At login, the server challenges you, and only the private key can produce the right answer, so the key itself never

crosses the network. That's why key-only login beats passwords (no guessing) and why private keys must be `600`.

REMEMBER

In the cloud, **cloud-init** (EC2 "user data") runs your setup script on first boot, and Session Manager often replaces SSH entirely. The Linux skills stay the same.

QUICK CHECK (answers at the back)

1. What does permission `640` mean?
2. Which command shows which ports a server is listening on?
3. Why does your private SSH key never cross the network during login?

3 How the web works: HTTP and APIs

> **Almost everything in the cloud talks over HTTP(S): websites, APIs, and even the AWS console and CLI, which are just HTTPS requests to AWS's APIs.**

A request and a response

```
GET /notes HTTP/1.1                ← method, path, version
Host: api.example.com              ← which site
Authorization: Bearer eyJhbGci...  ← who you are (a token)
Accept: application/json

HTTP/1.1 200 OK                     ← status code
Content-Type: application/json
[{"id": 1, "text": "hello"}]        ← body
```

Method	Means
GET	Read
POST	Create / send
PUT / PATCH	Replace / update
DELETE	Remove

Status	Means
2xx	Success (200 OK, 201 Created)
3xx	Redirect (301, 302)
4xx	Your mistake: 400 bad request, **401** not logged in, **403** not allowed, 404 not found, 429 too many requests
5xx	Server's problem (500, 502, 503)

REMEMBER

401 vs 403 is a favourite interview question. 401 = "I don't know who you are" (authentication failed). 403 = "I know who you are, and you're not allowed" (authorization failed).

APIs, JSON and YAML

- An **API** is a contract for how programs talk. A **REST API** uses URLs for things (`/notes/1`) and HTTP methods for actions.
- **JSON** is the data format of APIs and IAM policies: `{"key": "value", "list": [1, 2]}`.
- **YAML** is the same idea with indentation instead of brackets. It's used by GitHub Actions, Kubernetes, CloudFormation and Docker Compose. Indentation errors are the classic YAML bug.

Cookies, sessions and stateless HTTP

HTTP has no memory between requests. Websites remember you with a **session cookie** (a random ID the server looks up); APIs usually use a **token** sent with every request (chapter 9). Stealing either one lets an attacker act as you, which is why cookies are marked `Secure` (HTTPS only) and `HttpOnly` (JavaScript can't read them).

Try it

```
curl -i https://example.com                    # -i shows headers and status
curl -s https://api.example.com/notes | jq .   # jq formats JSON
```

QUICK CHECK (answers at the back)

1. What is the difference between a 401 and a 403?
2. Why do APIs send a token with every request?
3. What do the `Secure` and `HttpOnly` cookie flags do?

4 Code skills: Git, Bash, Python

> **Cloud engineers don't need to be software developers, but they must read and write small amounts of code every day: Terraform, scripts and pipelines, all stored in Git.**

Git: the model

Idea	Meaning
Repository	A project folder with its full history
Commit	A saved snapshot with a message
Branch	A separate line of work (e.g. `feature/waf`)
Remote	The copy on GitHub; `push` sends, `pull` fetches
Pull request	A request to merge a branch, where code is reviewed (and scanned in B4)

```
git status          # what changed?
git add FILE        # stage it
git commit -m "msg" # snapshot
git pull --rebase   # get others' changes first (you've done this)
git push
```

REMEMBER

Git never forgets. A secret committed once stays in the history even after you delete the file. If one ever slips in: treat it as leaked, rotate it immediately, then clean the history. Prevention: `.gitignore` and secret scanning.

Bash: glue for the command line

```
#!/usr/bin/env bash
set -euo pipefail                # stop on errors, unset variables and pipe failures
for region in eu-west-2 us-east-1; do
  echo "== $region =="
  aws ec2 describe-instances --region "$region" \
    --query 'Reservations[].Instances[].InstanceId' --output text
done
```

You already know `for` loops from your SSH tests. Add variables (`"$var"`, always quoted), `if` tests and exit codes (`0` = success).

Python: for anything bigger

```
import boto3                                    # the AWS SDK for Python
s3 = boto3.client("s3")
for b in s3.list_buckets()["Buckets"]:
    print(b["Name"])
```

Security teams use Python for automation: checking every bucket's settings, parsing logs and writing Lambda auto-responses (B6). Learn variables, lists, dictionaries, loops, functions, reading JSON and error handling (`try/except`).

QUICK CHECK (answers at the back)

1. You pushed a commit with an access key, then deleted it in the next commit. Is it safe? What do you do?
2. What does `set -euo pipefail` do in a Bash script?

5 Core security principles

> **Almost every security decision you'll make comes back to a handful of principles. Learn them well enough to name the principle behind any control.**

The CIA triad: what we protect

Goal	Means	Broken by	Protected by
Confidentiality	Only the right people see data	A public S3 bucket, a stolen laptop	Access control, encryption
Integrity	Data isn't changed without permission	Tampered logs, a modified file	Hashes, signatures, versioning
Availability	Systems work when needed	DDoS, ransomware, one failed server	Redundancy, backups, scaling

AAA: how access is controlled

- **Authentication:** who are you? (password + MFA)
- **Authorization:** what may you do? (IAM policy)
- **Accounting (auditing):** what did you do? (CloudTrail, Wazuh)

The principles

Principle	Meaning	Where you've used it
Least privilege	Give only the access needed, for only as long as needed	Firewall rules; the Wazuh read-only CloudTrail identity
Defence in depth	Several independent layers, so one failure isn't a breach	pfSense + ufw + SSH keys on the DMZ server
Default deny	Block everything not explicitly allowed	pfSense, IAM's implicit deny
Zero trust	Never trust based on network location; verify every request (identity, device, context)	Session Manager instead of "trusted" SSH from the LAN
Separation of duties	No single person can do something dangerous alone	Pull request review before Terraform applies
Secure by default	The default setting is the safe one	S3 Block Public Access on
Minimise attack surface	Fewer open ports, services and users mean fewer ways in	No public IPs; Docker bound to localhost
Fail securely	When something breaks, it breaks closed	If the authorizer errors, the API denies

THINK OF IT LIKE THIS

Defence in depth is a castle: a moat, walls, gates, guards and a locked keep. An attacker who crosses the moat still faces the walls. One cloud control (say, a security group) is the gate; the rest are still there if someone leaves it open.

QUICK CHECK (answers at the back)

1. A database is encrypted, but an attacker deletes it. Which part of CIA failed?
2. Name the principle: "the app server can reach the database, but nothing else can."
3. How is zero trust different from trusting everything inside the office network?

6 Risk and threat modelling

> **Security is about managing risk, not removing it. You can't fix everything, so you decide what matters most using a shared language.**

Term	Meaning	Example
Asset	Something of value	Customer database
Threat	Something that could cause harm	A criminal group
Vulnerability	A weakness that could be used	A public bucket, an unpatched server
Risk	The chance of harm × how bad it would be	High likelihood × high impact = fix first
Control	Something that reduces risk	Block Public Access, patching

Risk = Likelihood × Impact

A simple 1–5 scale for each gives a score out of 25 that you can rank

Four ways to treat a risk

- **Mitigate:** add a control (most common).
- **Transfer:** insurance or a provider takes it on.
- **Avoid:** stop doing the risky thing.
- **Accept:** the cost of fixing is higher than the risk. It must be a documented, owned decision, not ignorance.

Threat modelling with STRIDE

Before you build, ask "what could go wrong?" for each part of a design. STRIDE gives you six questions:

Threat	Breaks	Cloud example	Control
Spoofing	Authentication	Stolen access key used	MFA, short-lived credentials
Tampering	Integrity	Logs edited to hide an attack	Log file validation, Object Lock
Repudiation	Accountability	"I never deleted that"	CloudTrail
Information disclosure	Confidentiality	Public bucket	Block Public Access, encryption
Denial of service	Availability	Flood of requests	WAF rate limits, Shield, scaling
Elevation of privilege	Authorization	A role that can change its own permissions	Least privilege, permission boundaries

MITRE ATT&CK

A public catalogue of real attacker behaviour, organised as **tactics** (the goal, e.g. Credential Access) and **techniques** (how, e.g. T1110 Brute Force, which you've already mapped). There's a cloud matrix too. Defenders use it to check which techniques their detections cover.

QUICK CHECK (answers at the back)

1. What's the difference between a threat and a vulnerability?
2. Which STRIDE threat does CloudTrail mainly address?
3. When is accepting a risk the right choice?

7 Cryptography

> **Cryptography gives you three things: secrecy (encryption), proof that data wasn't changed (hashing), and proof of who sent it (signatures). Cloud key services such as AWS KMS are built from these pieces.**

Symmetric encryption: one shared key

The same key locks and unlocks. It's fast, so it's used for bulk data. The standard is **AES** (e.g. AES-256). The hard part is sharing the key safely.

Asymmetric encryption: a key pair

A **public key** (share it with anyone) and a **private key** (never share it). What one key locks, only the other unlocks. It's slower, so it's used for small things: exchanging keys and signing. Common algorithms: RSA and elliptic curve (ECC).

THINK OF IT LIKE THIS

Asymmetric encryption is a post box. Anyone can drop a letter through the slot (encrypt with the public key), but only the person with the key to the box can take letters out (decrypt with the private key).

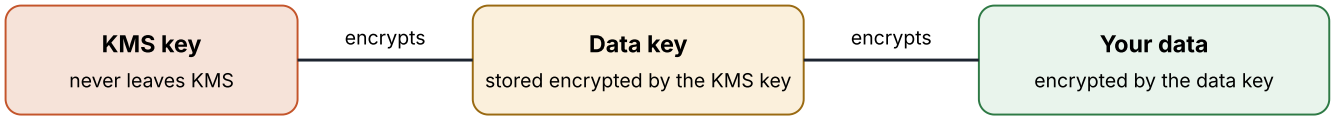
Hashing: a fingerprint

- A hash function (e.g. **SHA-256**) turns any data into a fixed-length fingerprint. The same input always gives the same hash; a one-letter change gives a completely different one; you can't reverse it.
- Uses: checking downloads and files haven't changed (integrity), and storing passwords.
- **Passwords** are stored with slow, **salted** hashes (bcrypt, scrypt, Argon2). The salt is random data added per user, so identical passwords get different hashes and precomputed tables don't work.
- **HMAC** = a hash mixed with a secret key, which proves the data came from someone who knows the key. AWS signs every API request this way (Signature Version 4).

Digital signatures

The sender hashes the data and encrypts the hash with their **private** key. Anyone can check it with the **public** key. A valid signature proves who sent it and that it wasn't changed. JWTs, software updates and certificates are all signed.

Envelope encryption: how KMS works



Your data is encrypted locally with a fast **data key**; the data key is itself encrypted by a **KMS key** that never leaves KMS. To read the data, you need permission to ask KMS to decrypt the data key. So

access to the key is access to the data, and every use is logged in CloudTrail.

State	Means	In AWS
Encryption at rest	Stored data is unreadable without the key	S3, EBS, RDS with KMS
Encryption in transit	Data moving over the network is protected	TLS (chapter 8)
In use	Protected even while being processed	Specialist hardware (Nitro Enclaves); rarely needed

QUICK CHECK (answers at the back)

1. Why use symmetric encryption for bulk data and asymmetric for key exchange?
2. Why are passwords salted before hashing?
3. In envelope encryption, what does an attacker need to read the data?

8 Certificates, PKI and TLS

> **TLS is what puts the "S" in HTTPS. It encrypts traffic and proves you're talking to the real server, using certificates signed by authorities your device already trusts.**

Certificates and the chain of trust

- A **certificate** binds a public key to a name (e.g. `shop.example.com`), signed by a **certificate authority (CA)**.
- Your browser and operating system ship with a list of trusted **root CAs**. Roots sign **intermediate** CAs, which sign websites' certificates. Checking each signature back to a trusted root is the **chain of trust**.
- The whole system of CAs, certificates and revocation is called **PKI** (public key infrastructure).
- Certificates expire, and expired certificates break sites. AWS Certificate Manager (ACM) issues and renews certificates for load balancers and CloudFront automatically.

The TLS 1.3 handshake, simplified

1. **Client hello:** "I support these ciphers; here's my half of a key exchange."
2. **Server hello:** the server picks a cipher, sends its half of the key exchange and its **certificate**, and proves it owns the matching private key.
3. The client **checks the certificate**: right name? Not expired? Chain leads to a trusted root?
4. Both sides now compute the same **session key** from the exchange (Diffie-Hellman). The rest of the conversation uses fast symmetric encryption.

REMEMBER

The session keys are temporary, so even if the server's private key is stolen later, recorded old traffic can't be decrypted. This is called **forward secrecy**. TLS 1.3 requires it.

Where TLS ends matters

With a load balancer, TLS often **terminates** at the load balancer, and traffic to the servers behind it may be plain HTTP inside the VPC. For sensitive data, re-encrypt from the load balancer to the servers too (end-to-end TLS). **Mutual TLS (mTLS)** goes further: the client also shows a certificate, so both sides are verified. It's common between services in zero-trust designs.

QUICK CHECK (answers at the back)

1. What does a CA's signature on a certificate prove?
2. What is forward secrecy?
3. What does "TLS terminates at the load balancer" mean for traffic behind it?

9 Authentication and authorization

> **Most cloud breaches begin with identity: a stolen password, a leaked key or a token. Knowing how logins and tokens work is the foundation of cloud security.**

Authentication factors

Factor	Examples	Note
Something you know	Password, PIN	Can be guessed, phished, reused
Something you have	Authenticator app, hardware key	Codes from an app can still be phished
Something you are	Fingerprint, face	Usually unlocks a device key

MFA combines two or more factors. **Phishing-resistant MFA** (FIDO2 security keys and passkeys) is strongest, because the key checks the website's real address and won't answer a fake site.

Tokens and JWTs

After login, systems give you a **token** instead of asking for your password every time. A **JWT** (JSON Web Token) has three parts separated by dots: `header.payload.signature`.

```
header:    {"alg": "RS256", "kid": "abc"}
payload:   {"sub": "user-123", "iss": "https://cognito...", "aud": "my-app", "exp":
1767225600}
signature: made with the issuer's private key
```

REMEMBER

A JWT is **signed, not encrypted**. Anyone can read the payload (it's only Base64-encoded), so never put secrets in it. The signature only proves it wasn't changed. APIs must check the signature, issuer, audience and expiry.

The protocols

Protocol	Answers	Where you'll see it
OAuth 2.0	**Authorization:** "can this app act on my behalf?" It issues access tokens.	"Allow this app to access your Google Drive"
OpenID Connect (OIDC)	**Authentication:** a login layer on top of OAuth that adds an ID token saying who the user is	"Sign in with Google", Cognito, Entra ID; GitHub Actions to AWS without keys
SAML 2.0	Single sign-on with signed XML	Older enterprise SSO into AWS and SaaS apps

Federation means trusting another system's logins. Your company identity provider (e.g. Entra ID) vouches for you, and AWS gives you a temporary role session. No separate AWS password exists to be stolen. IAM Identity Center works this way.

Authorization models

- **RBAC** (role-based): permissions come from your role (e.g. "Reader"). Simple; it can grow into too many roles.
- **ABAC** (attribute-based): permissions depend on attributes or tags, e.g. "you can manage resources tagged with your team". It scales better in the cloud.

QUICK CHECK (answers at the back)

1. Why can't you safely put a password inside a JWT payload?
2. OAuth 2.0 or OIDC: which one tells an app who the user is?
3. Why are security keys "phishing-resistant" when app codes aren't?

10 Common attacks

> **You can't defend against what you don't understand. These are the attacks behind most real incidents, with how each one looks in the cloud.**

Attack	How it works	Defence
Phishing	Tricking someone into giving credentials or running malware	Phishing-resistant MFA, training, email security (Guide 3 ch. 16)
Credential stuffing / brute force	Trying leaked or guessed passwords at scale	MFA, rate limits, detections (your rules 5712 and 100100)
Leaked secrets	Access keys in Git, logs or screenshots, found by automated scanners within minutes	No long-term keys, secret scanning, Secrets Manager
Misconfiguration	Public buckets, open security groups, over-broad IAM	Secure defaults, IaC scanning, posture tools (B4)
Injection (SQL, command)	Input treated as code: `' OR 1=1 --`	Parameterised queries, input validation, WAF
Cross-site scripting (XSS)	Injected JavaScript runs in other users' browsers	Output encoding, Content Security Policy, HttpOnly cookies
SSRF	Tricking a server into making requests for the attacker, for example to the instance metadata service to steal role credentials	**IMDSv2**, input validation, least-privilege roles
Supply chain	A compromised library, image or tool	Image scanning, pinned versions, trusted sources
Ransomware	Encrypting or stealing data for payment	Immutable backups (Object Lock), least privilege, detection
DDoS	Overwhelming a service with traffic	CDN, WAF rate limits, AWS Shield, auto scaling
Cryptojacking	Using stolen cloud access to mine cryptocurrency, which you pay for	Budget alerts, GuardDuty, SCPs limiting regions

The OWASP Top 10

The OWASP Top 10 is the standard list of the most critical web application risks. Broken access control has been at or near the top for years, followed by issues such as security misconfiguration, cryptographic failures, injection and vulnerable components. You practise several of them safely with DVWA and Juice Shop in your DMZ. Read the current edition on owasp.org.

THINK OF IT LIKE THIS

SSRF is convincing a hotel receptionist to read out the contents of the safe "because the manager asked". The receptionist (the server) has access you don't, and the attacker borrows it.

REMEMBER

In the cloud, the attacker's usual goal is **credentials**, not a server. A stolen key with broad permissions is worth more than a hacked VM. That's why identity is the perimeter (chapter 18).

QUICK CHECK (answers at the back)

1. How does IMDSv2 help defend against SSRF?
2. Why do attackers scan GitHub for access keys?
3. Which defence helps against both ransomware and accidental deletion?

11 Designing cloud systems

> **Good cloud design assumes things will fail, and builds so that failure is small, short and survivable. AWS's Well-Architected Framework is the vocabulary interviewers use.**

The six pillars of the Well-Architected Framework

Pillar	The question it asks
Operational excellence	Can we run, change and improve it safely (automation, runbooks)?
Security	Are identity, data, network and detection controls in place?
Reliability	Does it recover from failure and handle demand?
Performance efficiency	Are we using the right resources for the job?
Cost optimisation	Are we paying only for what delivers value?
Sustainability	Are we minimising environmental impact?

Availability, fault tolerance and disaster recovery

- **High availability:** the system stays up despite failures, usually by running in **two or more availability zones**.
- **Fault tolerance:** no interruption at all when a part fails. It's stronger and costs more.
- **Disaster recovery (DR):** getting back after a big event, such as a region outage, ransomware or a deleted account.

RPO = how much data can we lose? · RTO = how long can we be down?

Recovery Point Objective (e.g. 1 hour of data) · Recovery Time Objective (e.g. 4 hours to restore)

DR strategy	How it works	RTO/RPO	Cost
Backup and restore	Restore from backups when needed	Hours	Lowest
Pilot light	Core data replicated; servers off until needed	Tens of minutes	Low
Warm standby	A smaller copy always running, scaled up in a disaster	Minutes	Medium
Multi-site active/active	Full copies running in two places	Near zero	Highest

Design habits

Habit	Meaning
Scale **out**, not just up	Add more servers (horizontal) rather than a bigger one (vertical); no single point of failure
Stateless servers	Keep sessions and files elsewhere (a database, S3) so any server can be replaced
Loose coupling	Connect parts through queues and events, so one slow part doesn't stop the rest (chapter 15)
Treat servers as cattle, not pets	Replace broken servers from code instead of repairing them by hand
Managed services first	Let AWS run the database or queue; less for you to patch (shared responsibility)

REMEMBER

Backups are a security control as well as a reliability one. If an attacker can delete your backups with the same credentials they used on production, you don't have DR. Keep copies in another account and make them immutable.

QUICK CHECK (answers at the back)

1. What's the difference between RPO and RTO?
2. Why do stateless servers make auto scaling possible?
3. Which DR strategy keeps a scaled-down copy always running?

12 Cloud networking in depth

> **A VPC is your own private network inside AWS. Everything you learned with pfSense applies: subnets, routes, stateful rules and NAT. The cloud just gives them different names.**

Planning address space

- Pick a VPC range from private space (e.g. `10.40.0.0/16` = 65,536 addresses) that **doesn't overlap** any network you may connect later.
- AWS reserves **5 addresses** in every subnet (network, router, DNS, a future one, broadcast), so a `/24` gives 251 usable addresses.
- Each subnet lives in **one** availability zone. For high availability, create matching subnets in two or more AZs.

Getting traffic in and out

Component	Does	Home-lab twin
Route table	Decides where packets go; one per subnet	pfSense routing
Internet gateway	Two-way internet access for resources with public IPs	pfSense WAN
NAT gateway	Lets private resources reach *out* to the internet; nothing can start a connection in. Billed per hour.	pfSense outbound NAT
Security group	Stateful firewall on each network interface; allow rules only	pfSense rules
Network ACL	Stateless filter on a subnet; allow and deny	(none: pfSense is stateful)

Reaching AWS services privately: VPC endpoints

Type	Works for	How	Cost
Gateway endpoint	S3 and DynamoDB only	A route in your route table	Free
Interface endpoint (PrivateLink)	Most other services (SSM, KMS, Secrets Manager…)	A private network interface in your subnet	Hourly + per GB

Endpoints keep traffic off the internet and let you write policies such as "this bucket can only be reached through our endpoint".

Connecting networks

- **VPC peering:** a private link between two VPCs. It isn't transitive: if A peers with B and B with C, A still can't reach C.
- **Transit Gateway:** a central hub that many VPCs and VPNs attach to. It's the cloud version of the hub-and-spoke design in Guide 3.

- **Site-to-site VPN / Direct Connect:** link an office or data centre to AWS, over the internet (VPN) or a private line (Direct Connect).

DNS, load balancers and CDNs

- **Route 53** is AWS's DNS. *Private hosted zones* give names that work only inside your VPCs.
- **Load balancers:** an **Application Load Balancer** works at layer 7 (it understands HTTP paths and headers, and integrates with WAF). A **Network Load Balancer** works at layer 4 (TCP/UDP: very fast, with fixed IPs).
- **CloudFront** is a CDN: copies of content cached at edge locations near users. It's faster, absorbs attacks, and is the place to attach WAF.

QUICK CHECK (answers at the back)

1. How many usable addresses does a `/24` subnet give in AWS, and why?
2. A private instance needs to download updates. What lets it reach out without being reachable from the internet?
3. Why is VPC peering described as non-transitive?

13 Compute: VMs, containers, serverless

> **The three ways to run code in the cloud trade control for convenience. The more AWS manages, the less you patch, and the less you can customise.**

	VMs (EC2)	Containers (ECS/EKS, Fargate)	Serverless (Lambda)
You manage	OS, patches, runtime, app	Image and app (and hosts unless you use Fargate)	Code and its permissions
Starts in	Minutes	Seconds	Milliseconds (plus "cold starts")
You pay for	Running time	Running time	Each request and its duration
Good for	Legacy apps, full control	Microservices, portability	Events, APIs, automation

EC2 essentials

- **Instance types** are grouped into families: general purpose (t, m), compute (c), memory (r). `t` types are cheap and "burstable".
- **AMI:** the template image an instance starts from. Harden it once and reuse it ("golden image").
- **EBS:** a network disk attached to the instance. Encrypt it, and snapshot it for backups or forensics.
- **User data:** a script that runs on first boot. Never put secrets in it, because anyone who can view the instance's settings can read it.
- **Instance profile (role):** how an instance gets temporary AWS credentials, so there are no keys on disk.
- **Instance metadata service (IMDS):** a local address (`169.254.169.254`) where the instance reads its role credentials. Require **IMDSv2**, which needs a session token obtained with a PUT request. That blocks most SSRF attacks (chapter 10).
- **Pricing options:** On-Demand (flexible), Savings Plans/Reserved (commit and save), Spot (spare capacity, very cheap, can be taken back with 2 minutes' notice).

Containers

A container image packages an app with its dependencies (Guide 2). **ECR** stores images and scans them; **ECS** is AWS's own scheduler; **EKS** is managed Kubernetes; **Fargate** runs containers without you managing servers. Each ECS task gets its own **task role**, so permissions are per app rather than per host.

Lambda

- Code runs in response to an **event**: an HTTP request, a file landing in S3, a message, a schedule or a GuardDuty finding.
- Limits to remember: up to **15 minutes** per run; memory is configurable (more memory also gives more CPU).
- Its **execution role** is its identity. That's where least privilege matters most.

QUICK CHECK (answers at the back)

1. Why shouldn't secrets go in EC2 user data?
2. What does IMDSv2 require that IMDSv1 didn't?
3. Who patches the operating system under a Lambda function?

14 Storage and databases

> **Choosing the right storage is about how data is accessed. Securing it is mostly about who can reach it, whether it's encrypted, and whether you can recover it.**

Type	Like	AWS	Use for
Block	A hard drive	EBS	An instance's disk, databases
File	A shared network drive	EFS (Linux), FSx	Files shared by many servers
Object	An unlimited bucket of files with metadata	S3	Backups, logs, websites, data lakes

S3 in depth

- Objects live in **buckets**, addressed by key (`logs/2027/01/file.json`). Bucket names are globally unique.
- **Strong read-after-write consistency:** once a write succeeds, reads see it immediately.
- **Storage classes** trade price for access speed: Standard → Infrequent Access → Glacier (archive). **Lifecycle rules** move or delete objects automatically.
- **Versioning** keeps old versions, protecting against overwrites and deletion. **Object Lock** makes versions undeletable for a set period (WORM: write once, read many), which is ransomware-proof backup.
- **Access:** bucket policies, IAM policies, Block Public Access (keep it on), and Object Ownership set so ACLs are disabled.

Databases

	Relational (SQL)	NoSQL (key-value / document)
Shape	Tables with fixed columns and relationships	Flexible items looked up by key
AWS	RDS (PostgreSQL, MySQL…), Aurora	DynamoDB
Scales by	Bigger instances, read replicas	Automatically, almost without limit
Good for	Transactions, complex queries	High-traffic, simple access patterns

REMEMBER

Multi-AZ vs read replicas is a classic exam question. **Multi-AZ** keeps a standby copy in another AZ with synchronous replication, for **availability** (automatic failover). **Read replicas** are asynchronous copies for **scaling reads**.

DynamoDB tables need a **partition key** (and optionally a sort key). Design around how you'll query, because you can't join tables. **Caching** (ElastiCache, or DAX for DynamoDB) keeps hot data in memory to cut load and latency.

QUICK CHECK (answers at the back)

1. Which S3 feature makes backups undeletable for a set time?
2. You need automatic failover for an RDS database. Multi-AZ or a read replica?
3. Why must you design DynamoDB tables around your queries?

15 Connecting services: queues and events

> **When services call each other directly, one failure spreads. Putting a queue or an event bus in between lets each part work at its own pace and fail on its own.**

Pattern	AWS	How it works
Queue (point to point)	SQS	Producers put messages in; consumers take them out when ready. Each message is handled by one consumer.
Pub/sub (fan-out)	SNS	One message is pushed to many subscribers (email, Lambda, queues)
Event bus	EventBridge	Events from AWS services and apps are matched by rules and routed to targets
Workflow	Step Functions	Coordinates several steps with retries and branches

Ideas to know

- **Synchronous vs asynchronous:** a synchronous call waits for an answer (HTTP API). An asynchronous one drops a message and moves on.
- **SQS Standard vs FIFO:** Standard is massively scalable, delivers **at least once**, and order is best-effort. FIFO keeps strict order and avoids duplicates, at lower throughput.
- **Idempotency:** processing the same message twice has the same effect as once. Needed because "at least once" means duplicates can happen.
- **Dead-letter queue (DLQ):** where messages go after failing several times, so you can inspect them instead of losing them.

IN YOUR LAB

Your B6 auto-response is event-driven: **GuardDuty finding → EventBridge rule → Lambda → isolate the instance**. Security automation in the cloud is almost always built this way.

QUICK CHECK (answers at the back)

1. Why does a queue make a system more resilient?
2. What is idempotency, and why does SQS Standard make it necessary?
3. Which service would route a GuardDuty finding to a Lambda function?

16 Infrastructure as code and CI/CD

> **Infrastructure as code (IaC) describes your environment in files; CI/CD runs checks and deploys those files automatically. Together they make cloud changes reviewable, repeatable and secure.**

Declarative vs imperative

Imperative (CLI scripts) says *how*: "create a VPC, then a subnet". **Declarative** (Terraform, CloudFormation) says *what*: "there should be a VPC with these subnets", and the tool works out the steps. Running declarative code twice changes nothing the second time. That's **idempotent**, just like in chapter 15.

Terraform's model

Concept	Meaning
Provider	The plugin that talks to an API (aws, azurerm)
Resource	One thing to manage (`aws_s3_bucket`)
Data source	Something to read but not manage (e.g. the latest AMI)
State	Terraform's record of what exists. It can contain secrets, so store it encrypted, remotely, with locking.
Plan	The difference between the code and reality, shown before any change
Module	A reusable package of resources (your VPC module)
Drift	Reality changed outside the code (someone clicked in the console). `plan` reveals it.

A secure pipeline (B4)



- **Shift left:** catch problems as early (as far "left") in the pipeline as possible. A public bucket blocked in a pull request costs seconds; found in production, it might be a breach.
- **Pipeline credentials:** the pipeline should get short-lived AWS credentials through **OIDC federation**, not stored access keys. The pipeline is a powerful identity and a favourite target.
- **Policy as code:** rules such as "no public buckets" written as code and enforced automatically (Checkov rules, SCPs, Config rules).

QUICK CHECK (answers at the back)

1. What is drift, and how do you detect it?
2. Why should a CI/CD pipeline use OIDC instead of stored access keys?
3. What does "shift left" mean?

17 Observability: logs, metrics, traces

> **You can't secure or fix what you can't see. Observability means collecting the right signals, keeping them safe, and turning them into alerts someone acts on.**

Signal	Answers	Example
Logs	What happened, in detail?	"User X called DeleteBucket at 10:02"
Metrics	How much, how often, how fast?	CPU 90%, 500 errors per minute
Traces	Where did one request spend its time across services?	API → Lambda → DynamoDB (AWS X-Ray)

The AWS logging map

Source	Records	Security use
CloudTrail	API calls: who did what, when, from where. *Management events* (changing resources) are on by default; *data events* (e.g. reading S3 objects) must be enabled and cost extra.	The core audit trail; investigations
CloudWatch Logs	Application and OS logs, Lambda output	App-level attacks, errors
VPC Flow Logs	Network connections (IPs, ports, accept/reject). No packet contents.	Scans, data exfiltration, unexpected traffic
AWS Config	The configuration history of resources	"When did this bucket become public?"
Load balancer / CloudFront / WAF logs	Web requests and blocks	Web attacks

From signal to action

Collect → store centrally (a separate logging account, immutable) → analyse (a SIEM such as Wazuh or Sentinel, or Athena queries) → alert → respond. This is the pipeline you built on-prem: agent → manager → indexer → dashboard. In Case 001 you saw what happens when one step (time) is wrong.

REMEMBER

Protect the logs themselves. Attackers try to stop CloudTrail or delete logs to hide. Defences: an organisation trail that member accounts can't turn off, log file validation, a separate log archive account, and an alert on `StopLogging`.

QUICK CHECK (answers at the back)

1. What's the difference between CloudTrail management events and data events?
2. Do VPC Flow Logs contain packet contents?
3. Why store logs in a separate account?

18 Identity is the perimeter

> **In the cloud, anyone with valid credentials can reach the API from anywhere, so the network edge no longer protects you. Identity and permissions are the real boundary.**

How AWS decides: policy evaluation, simplified

1. Everything starts as **denied** (implicit deny).
2. If any applicable policy says **Deny** → **denied**. An explicit deny always wins.
3. Guardrails must allow it: **SCPs** from AWS Organizations, **permission boundaries** and session policies. These never grant access; they only set the maximum.
4. Then an **identity policy** (attached to the user or role) or a **resource policy** (attached to the bucket, key or queue) must **Allow** it.

THINK OF IT LIKE THIS

Think of a building. The SCP is the building's rules ("nobody enters the roof"). The permission boundary is your contract ("you'll never get more than floors 1–3"). Your badge (identity policy) opens specific doors. Any "no entry" sign (explicit deny) beats every badge.

Identity policies vs resource policies

	Identity policy	Resource policy
Attached to	A user, group or role	A resource (S3 bucket, KMS key, SQS queue…)
Says	"This identity can do X to Y"	"These principals can do X to me"
Cross-account	Not by itself	Can grant access to other accounts

Roles and temporary credentials

- A **role** is assumed through **STS** (Security Token Service), which returns credentials that expire (minutes to hours). Stolen temporary credentials soon stop working; stolen long-term access keys work until someone notices.
- Each role has a **trust policy** saying who may assume it. A badly written trust policy (e.g. any AWS account) is a classic hole.
- **Cross-account access:** account A's role trusts account B. That's how central security tooling reaches every account.

Multi-account strategy

Companies use **many accounts** (production, development, security, log archive) grouped in **AWS Organizations**. An account is the strongest isolation boundary in AWS: a mistake in development can't touch production. **SCPs** apply guardrails to whole groups of accounts, for example "no one may disable CloudTrail" or "only these regions".

REMEMBER

Common identity mistakes: `"Action": "*"` on `"Resource": "*"`; long-lived access keys; no MFA; unused users never removed; trust policies that trust too broadly. IAM Access Analyzer finds external access and unused permissions.

QUICK CHECK (answers at the back)

1. An identity policy allows `s3:DeleteObject`, but an SCP denies it. What happens?
2. Can a permission boundary grant access?
3. Why are temporary credentials safer than access keys?

19 Securing data and workloads

> **Protect data according to how sensitive it is, and harden every workload so a single bug or stolen credential doesn't become a breach.**

Data protection

1. **Know your data:** classify it (public, internal, confidential, personal). Amazon Macie finds personal data in S3. Personal data brings legal duties (GDPR, NDPA: chapter 21).
2. **Control access** with least-privilege IAM, resource policies and Block Public Access.
3. **Encrypt** at rest with KMS (chapter 7) and in transit with TLS (chapter 8).
4. **Key policies:** every KMS key has its own resource policy. Separate who *administers* keys from who *uses* them (separation of duties).
5. **Secrets** (database passwords, API keys) belong in Secrets Manager or Parameter Store, fetched at runtime, rotated, never in code or images.
6. **Back up** and test restores; keep immutable copies (Object Lock, AWS Backup vault lock).

Workload protection

Control	Why
Hardened images (AMIs, container base images)	A known-good starting point; fewer packages mean fewer vulnerabilities
Patching (Systems Manager Patch Manager)	Most compromises use known, fixable vulnerabilities
Vulnerability scanning (Amazon Inspector, ECR scans)	Find vulnerable packages on instances, images and Lambda
IMDSv2 required	Blocks SSRF credential theft
No inbound admin ports; Session Manager	Smaller attack surface, every session logged
Least-privilege roles per workload	A compromised app can only do what its role allows
Immutable infrastructure	Replace instead of patching by hand; rebuild from trusted code after an incident

QUICK CHECK (answers at the back)

1. Why separate key administrators from key users?
2. Where should an application's database password live?
3. Name two controls that limit damage after an application is compromised.

20 Detection and response in the cloud

> **Prevention eventually fails. Detection finds the attack quickly; response limits the damage. In the cloud, both can be largely automated.**

Types of detection

Type	Finds	Example
Signature / rule	Known bad patterns	Your rule 5710: "invalid user"
Correlation / threshold	Many events that together mean something	Rules 5712 and 100100
Anomaly / behaviour	Unusual activity compared with normal	GuardDuty: API calls from a new country or a known-bad IP
Threat intelligence	Known malicious IPs, domains, hashes	GuardDuty threat lists
Posture (CSPM)	Risky configuration rather than activity	Security Hub: "bucket is public"

High-value cloud detections

- Root user activity; console login without MFA.
- `StopLogging` / `DeleteTrail`; GuardDuty or Config being disabled.
- New access keys, especially for privileged users; IAM policy changes.
- Security group opened to `0.0.0.0/0` on admin ports; buckets made public.
- Activity in regions you never use (a cryptojacking sign).

Incident response in the cloud

The phases from Guide 3 ch. 14 (preparation; detection and analysis; containment, eradication and recovery; lessons learned) still apply. The 2025 revision of NIST SP 800-61 maps them to the NIST CSF 2.0 functions. What changes in the cloud is **how** you act:

Step	Cloud version
Contain a compromised instance	Swap its security group for an "isolation" group with no rules; detach its role
Contain a leaked key	Deactivate the key; revoke the role's active sessions
Preserve evidence	EBS snapshots, memory capture before shutdown, copy logs to a locked bucket
Investigate	CloudTrail, Flow Logs, Detective; build a timeline
Recover	Rebuild from trusted code and images rather than cleaning the old instance

REMEMBER

Prepare before the incident: an IR role in every account, logging already on, playbooks written and practised. You can't turn on CloudTrail afterwards to see what happened yesterday.

QUICK CHECK (answers at the back)

1. Name two detection types and one example of each.
2. How do you contain a compromised EC2 instance without destroying evidence?
3. Why is "rebuild from code" better than "clean the server"?

21 Governance and compliance

> **Governance is how an organisation decides and proves that it's secure enough. Frameworks give a shared checklist; automation keeps it true every day rather than once a year.**

The frameworks you'll hear about

Framework	What it is
NIST CSF 2.0	Six functions: **Govern, Identify, Protect, Detect, Respond, Recover**. A common way to organise a security programme.
ISO/IEC 27001	An international, certifiable standard for an information security management system
SOC 2	An audit report on a service provider's controls, often requested by customers of SaaS companies
CIS Benchmarks	Detailed secure-configuration checklists (e.g. the CIS AWS Foundations Benchmark in Security Hub)
PCI DSS	Rules for anyone handling card payments
GDPR / UK GDPR	Personal data protection law in the EU and UK: lawful basis, minimisation, breach notification within 72 hours
NDPA 2023	Nigeria's data protection law, enforced by the NDPC

Cloud governance in practice

- **Shared responsibility** (Guide 3 ch. 4) decides which controls are yours to prove. A provider's certificate covers their part, not your configuration.
- **Guardrails:** preventive (SCPs, IAM, Block Public Access: stop it happening) and detective (Config rules, Security Hub: tell you it happened).
- **Posture management (CSPM):** continuously checking every account against a benchmark. Security Hub and Prowler do this in B4.
- **Evidence:** auditors want proof. Logs, Config history and IaC in Git make proof automatic.
- **Tagging and ownership:** every resource has an owner, so findings reach someone who can fix them.

REMEMBER

Compliance isn't security. You can pass an audit and still be breached. But a good framework is a useful map of what to cover, and many employers ask how you'd apply one.

QUICK CHECK (answers at the back)

1. Name the six functions of NIST CSF 2.0.
2. What's the difference between a preventive and a detective guardrail?
3. Does using AWS make your application compliant automatically?

22 Glossary and answers

Glossary

Term	Meaning
ABAC / RBAC	Access decided by attributes (tags) / by role
AMI	Amazon Machine Image: the template an EC2 instance starts from
AZ	Availability zone: one or more separate data centres in a region
CIA	Confidentiality, integrity, availability
CSPM	Cloud security posture management: continuous configuration checks
DLQ	Dead-letter queue for messages that keep failing
Drift	Real infrastructure no longer matches the code
Envelope encryption	Data encrypted with a data key, which is encrypted with a master key
Federation	Trusting another identity provider's logins
Forward secrecy	Old traffic stays safe even if a long-term key leaks later
Idempotent	Doing it twice has the same effect as doing it once
IMDSv2	The token-based instance metadata service that blocks most SSRF credential theft
JWT	A signed token: header.payload.signature
KMS	Key Management Service: creates and controls encryption keys
mTLS	TLS where both client and server present certificates
OAuth 2.0 / OIDC	Delegated authorization / login on top of it
PKI	The system of certificate authorities and certificates
RPO / RTO	How much data you can lose / how long you can be down
SCP	Service control policy: a guardrail on accounts in AWS Organizations
Shift left	Catching problems earlier in development
SSRF	Server-side request forgery: making a server send requests for the attacker
STRIDE	Spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege
STS	Security Token Service: issues temporary credentials
Zero trust	Verify every request; never trust because of network location

Answers

1. (1) The kernel controls hardware and runs with full privileges; user space is where normal programs run with limited rights and ask the kernel for access. (2) If the service is compromised, the attacker gets

only that user's limited rights instead of root.

2. (1) Owner read and write; group read; others nothing. (2) `ss -tulpn`. (3) The server sends a challenge that only the private key can answer correctly; only the answer travels.

3. (1) 401: not authenticated (who are you?); 403: authenticated but not allowed. (2) HTTP is stateless: the server doesn't remember earlier requests, so each one must prove who sent it. (3) `Secure`: sent only over HTTPS; `HttpOnly`: JavaScript can't read it, which limits theft through XSS.

4. (1) No. It's still in the history and may already have been scraped. Deactivate and rotate the key immediately, check CloudTrail for use, then clean the history. (2) Stops the script on any error, on unset variables, and on failures inside pipes.

5. (1) Availability. (2) Least privilege (also minimising attack surface). (3) Zero trust verifies each request's identity and context instead of trusting anything on the internal network.

6. (1) A threat is what could cause harm; a vulnerability is the weakness it could use. (2) Repudiation. (3) When fixing costs more than the risk, and the decision is documented and owned by someone with authority.

7. (1) Symmetric is fast but needs a shared key; asymmetric solves sharing but is slow, so it's used to exchange or protect the symmetric key. (2) So identical passwords get different hashes and precomputed lookup tables are useless. (3) Permission to use the KMS key to decrypt the data key (plus access to the encrypted data).

8. (1) That the CA checked the certificate holder controls that name, and the certificate hasn't been changed. (2) Session keys are temporary, so a later leak of the server's key can't decrypt old recorded traffic. (3) Traffic is decrypted at the load balancer; behind it, it may travel unencrypted unless you re-encrypt.

9. (1) The payload is only encoded, so anyone holding the token can read it. (2) OIDC. (3) The key checks the real website address and won't sign in to a look-alike site; a typed code can be relayed by a fake site.

10. (1) It requires a session token obtained with a PUT request, which most SSRF bugs can't make. (2) Leaked keys give direct access to cloud accounts and are found automatically within minutes. (3) Immutable, versioned backups (e.g. S3 Object Lock).

11. (1) RPO is how much data you can afford to lose; RTO is how long you can afford to be down. (2) Any server can be added or removed because no user data lives on it. (3) Warm standby.

12. (1) 251, because AWS reserves 5 addresses in every subnet. (2) A NAT gateway (or VPC endpoints for AWS services). (3) Traffic can't pass through a middle VPC: A↔B and B↔C doesn't give A↔C.

13. (1) Anyone who can view the instance's configuration can read it, and it often ends up in logs. (2) A session token, fetched with a PUT request and sent in a header. (3) AWS.

14. (1) Object Lock. (2) Multi-AZ. (3) It has no joins and is queried by key, so the key design decides which questions you can ask efficiently.

15. (1) The producer and consumer work independently; if the consumer fails or slows down, messages wait safely. (2) Processing a message twice has the same effect as once; Standard queues can deliver duplicates. (3) EventBridge.

16. (1) Real resources changed outside the code; `terraform plan` shows the difference. (2) Short-lived credentials can't leak like stored keys and expire quickly. (3) Finding problems earlier, when they're

cheaper to fix.

17. (1) Management events record changes to resources (on by default); data events record actions on data, such as reading S3 objects (opt-in, extra cost). (2) No: only connection details. (3) So someone who compromises a workload account can't delete the evidence.

18. (1) Denied: the SCP's explicit deny wins. (2) No: it only sets the maximum permissions. (3) They expire automatically, so stolen ones stop working soon.

19. (1) So no single person can both change a key's rules and use it to read data. (2) Secrets Manager (or Parameter Store), fetched at runtime. (3) Any two of: least-privilege role, IMDSv2, network segmentation, no stored secrets, monitoring.

20. (1) For example: rule-based (5710 invalid user) and anomaly-based (GuardDuty unusual API location). (2) Apply an isolation security group, detach its role, snapshot its disks, and capture memory if possible, without terminating it. (3) Cleaning may miss a backdoor; a rebuild from trusted code is known-good.

21. (1) Govern, Identify, Protect, Detect, Respond, Recover. (2) Preventive stops the action (SCP); detective reports it after it happens (Config rule). (3) No: AWS secures its part; your configuration, data and access are still your responsibility.